Quantitative Trading Research Platform

Methodology, Architecture, and Engineering Decisions

Salman Linjawi

linjakid@gmail.com github.com/Linja1337/quant-research-platform

May 2026

1. Summary

A hybrid C# / Python platform for designing systematic futures and crypto strategies, validating them with statistical machinery from the academic literature (CSCV, PBO, DSR, walk-forward, permutation tests), and reconciling execution semantics across multiple backtesting frameworks before any live deployment. The public repository is a sanitized showcase of the architecture, the validation methodology, runnable synthetic-data demonstrations of every technique, and the engineering decisions behind the stack.

The platform's premise is that any backtest is overfit until proven otherwise. The validation gates exist to shoulder the burden of proof. Each gate is designed to catch failure modes that the others miss: walk-forward catches IS-to-OOS degradation, CSCV catches selection bias from picking the IS-best, DSR catches multiple-testing inflation, permutation tests catch market drift mistaken for signal, parameter stability catches knife-edge fits, and parity audits catch implementation bugs. A candidate must clear all six to be considered deployment-eligible. Selectively waiving a gate is explicitly disallowed.

1.1 Architecture in one paragraph

The C# layer (running on a commercial trading platform) handles live order routing and is the execution authority. The Python layer (NumPy, pandas, scipy, Optuna, DEAP, Numba) handles research, optimization, and validation. A custom Numba-compiled engine runs roughly a thousand backtests per second on a laptop, which is what makes 200-trial Optuna sweeps and 80-configuration CSCV grids feasible without a cluster. Cross-framework parity audits reconcile the C# strategy against three independent Python frameworks (Backtrader, backtesting.py, vectorbt). A Markdown decision log carries memory across sessions and constrains AI tooling to operate inside a deterministic envelope.

1.2 What this brief contains

Page 2 walks through the validation methodology with the formal definitions and operational thresholds. Page 3 walks through the engineering decisions and what was rejected. The full repository (architecture, methodology, demos, and recruiter-facing summaries by role) is at the GitHub URL above.

2. Methodology

2.1 Walk-forward validation (Pardo 2008)

In-sample windows of 60 calendar days are used to fit parameters; the immediately following 20-day out-of-sample windows score them with no re-optimization. Walk-forward efficiency per fold is $\mathrm{WFE} = \mathrm{OOS}_{\mathrm{metric}}/\mathrm{IS}_{\mathrm{metric}}$. Operational target: $\mathrm{WFE} > 0.50$ with a tight distribution across folds. Bimodal WFE usually means the strategy depends on a regime present in some folds and not others.

2.2 Combinatorially Symmetric Cross-Validation (Bailey et al. 2014)

Build a performance matrix M of shape (T, N) across T time blocks and N strategy configurations. Enumerate all $\binom{T}{T/2}$ symmetric partitions of the T blocks into in-sample and out-of-sample halves. For each partition, pick the IS-best configuration n^* and compute its rank quantile $\bar{\omega}$ in the OOS distribution. The Probability of Backtest Overfitting is

$$\mathrm{PBO} = \mathbb{P}\left[\ln \frac{\bar{\omega}}{1-\bar{\omega}} \leq 0\right].$$

Operational target: $\mathrm{PBO} < 0.10$. PBO measures the selection process, not any individual strategy. A pure-noise population of strategies reliably produces $\mathrm{PBO} \approx 0.5$, which is the cleanest sanity check on the implementation.

2.3 Deflated Sharpe Ratio (Bailey & Lopez de Prado 2014)

Expected maximum Sharpe under the null hypothesis (no edge) for N independent trials, by the False Strategy Theorem:

$$\mathrm{SR}_0 = \sqrt{\mathbb{V}[\hat{\mathrm{SR}}_n]} \cdot \left[(1-\gamma)\,\Phi^{-1}\left(1-\frac{1}{N}\right) + \gamma\,\Phi^{-1}\left(1-\frac{1}{Ne}\right)\right],$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant, $e \approx 2.718$, and Φ^{-1} is the inverse standard-normal CDF. The deflated Sharpe is

$$\mathrm{DSR} = \Phi\left(\frac{(\mathrm{SR}_{\mathrm{obs}} - \mathrm{SR}_0)\sqrt{T-1}}{\sqrt{1-\hat{\gamma}_3\,\mathrm{SR}_{\mathrm{obs}} + \frac{\hat{\gamma}_4 - 1}{4}\,\mathrm{SR}_{\mathrm{obs}}^2}}\right),$$

with $\hat{\gamma}_3$ and $\hat{\gamma}_4$ the sample skewness and kurtosis of the returns. Operational target: $\mathrm{DSR} > 0.95$. The expected maximum Sharpe under the null grows roughly with $\sqrt{2\ln N}$, so at $N = 1{,}000$ trials a Sharpe of 3.7 is the threshold below which a result is statistically indistinguishable from luck.

2.4 Permutation tests

Day-block permutation shuffles the order of trading days while preserving intraday bar order, destroying inter-day temporal structure but preserving intraday autocorrelation. Resulting null distribution is the strategy's Sharpe under the hypothesis "the inter-day evolution is permutable." p-value is the fraction of permutations whose Sharpe exceeds the observed Sharpe. Operational target: $p < 0.05$.

2.5 Parameter stability

For each parameter, perturb the optimum by ± 1 and ± 2 grid steps. Run a lightweight CSCV on the perturbed configurations and record the mean OOS PnL per neighbor. The stability score is the fraction of one-step neighbors with positive mean OOS PnL. Operational target: ≥ 0.80. Plateau parameter regions generalize; knife-edge regions do not.

2.6 Cross-framework parity audits

The same strategy is implemented in the C# execution authority and in three independent Python frameworks. Trade lists are diffed trade-by-trade. The most-cited bug class this catches is implicit unit mismatches: futures multipliers, tick values, percent-vs-absolute commissions, share-vs-notional quantities. Each silently scales the answer without raising an error. A strategy is not eligible for paper trading until at least two frameworks agree on its trade list.

3. Engineering decisions

3.1 Live execution: C# / .NET (constrained, not preferred)

The commercial trading platform exposes a strategy SDK in C#, so any strategy that runs live runs in C#. The C# layer is single-threaded per backtest and slow for parameter sweeps, which motivates the separate Python research stack.

3.2 Research stack: Python

NumPy, pandas, scipy, Optuna, DEAP, Numba, and matplotlib cover almost every numerical and statistical need with no commercial license cost. R (weaker general-purpose ecosystem), Julia (smaller ecosystem, weaker AI tooling), and C++ (development velocity cost too high for research code that may change weekly) were considered and rejected.

3.3 Engine: custom Numba JIT, with off-the-shelf frameworks for parity

A custom Numba-compiled engine runs roughly a thousand backtests per second on a laptop and models exactly the execution semantics that the live C# strategy uses. Off-the-shelf engines either run slower (backtesting.py) or have execution models that diverge from C# in subtle ways (vectorbt's vectorized fills, Backtrader's broker state machine). Vectorbt and Backtrader implementations exist, but as parity checks against the primary engine, not as the engine itself.

3.4 Search: Optuna TPE primary, DEAP genetic fallback

TPE is sample-efficient and handles mixed continuous-discrete spaces. DEAP exists for spaces with discontinuous valid regions (constraints like fast $<$ slow that prune entire grid sections) where TPE struggles. Grid and random search were rejected for the obvious reasons.

3.5 Decision memory: Markdown decision log under git

Every hypothesis is logged with hypothesis, change, expected outcome, actual result, verdict, and (if falsified) the parameter region marked as a forbidden region. A Markdown file is tool-agnostic, diff-able, and survives any change in research tooling. Notion, SQLite, per-iteration JSON files, and platform wikis were considered and rejected: each couples the research log to a specific tool.

3.6 Demo data: synthetic only

The methodology this repository demonstrates is general; the strategies that exercise it are not. Synthetic data is reproducible by anyone, runs without an API key, and contains no identifying information about the private platform's parameter choices or live performance. Public-domain bar data was rejected because a competitor who knows the strategy class could re-run the public demo on the real underlying and back into the platform's parameters.

3.7 What is still open

The pure-Python execution stack (bypassing C# for direct broker connectivity), GPU array math for larger sweeps, and streaming bar ingestion that unifies the research and live code paths are all unfinished. Listed because a portfolio that claims everything works is less credible than one that names what does not.

Built by Salman Linjawi. Email: `linjakid@gmail.com`. Repository: `github.com/Linja1337/quant-research-platform`.